



Curso Avanzado de Python con Django



Sesiones:

01

Fundamentos de DJANGO

02

Modelos y Base de datos

03

Vistas, URLs y plantillas

04

Autenticación y seguridad

05

API REST con DRF

06

Despliegue y proyecto final

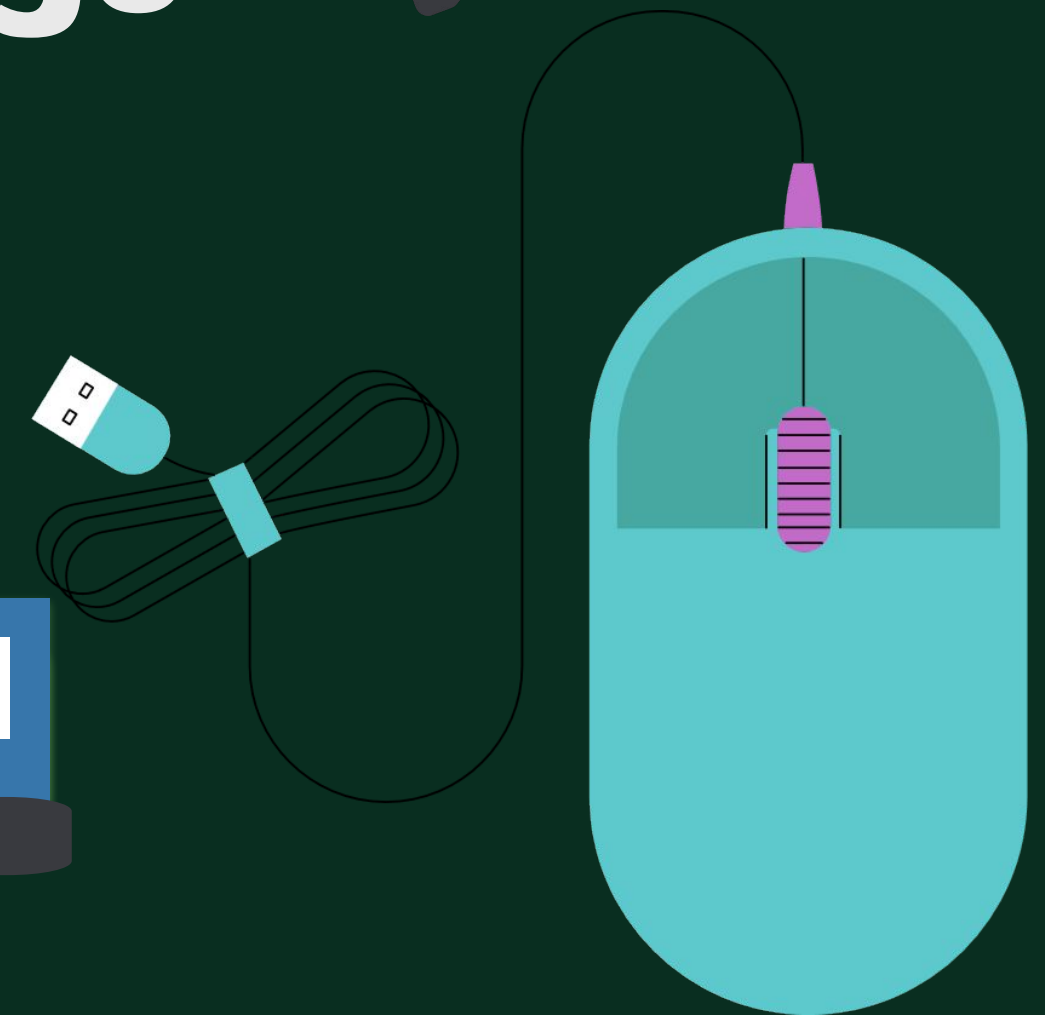


01

Fundamentos de Django



Tu Primer desarrollo web Profesional



Objetivos:

-  **Comprender Django y MTV**
-  **Configurar entorno**
-  **Crear primer proyecto**
-  **Ejecutar servidor**

¿Qué es Django?

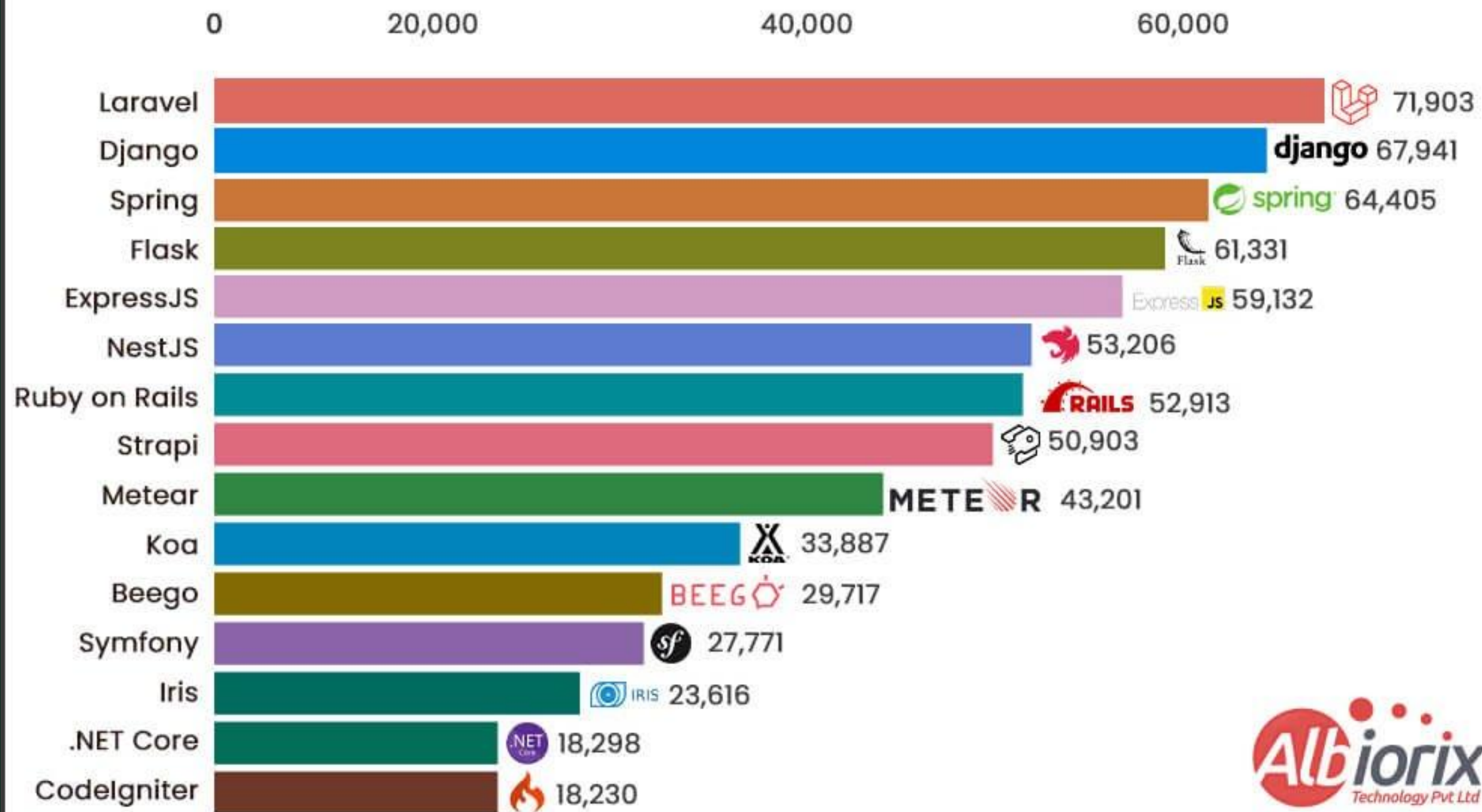
Django es un popular framework web de alto nivel escrito en Python que permite a los desarrolladores crear aplicaciones web robustas rápidamente.

Django incluye un sistema ORM (Mapeo Relacional de Objetos) que simplifica las interacciones con bases de datos y la creación de esquemas.

django



Most Popular Backend Frameworks



Los datos del vídeo se calculan en función de las estrellas del archivo de GitHub. Pero en el artículo veremos y analizaremos también otros parámetros. **Frameworks de backend más populares – 2012/2023.**



¿QUÉ ES **django**?

Es uno de los **frameworks** más populares de **Python** para el desarrollo de **aplicaciones web** y usa el **patrón MVC**.



CARACTERÍSTICAS

ADMINISTRADOR

Cuenta con un panel de administración que permite **crear, consultar, actualizar y eliminar** contenido.



BASES DE DATOS

Tiene un **ORM** que se encarga de la comunicación con la base de datos. Es compatible con **PostgreSQL, MySQL, SQLite, etc.**



SEGURIDAD

Tiene opciones de protección contra ataques de **SQL injection** o **XSS** (cross site scripting).



EMPRESAS QUE USAN DJANGO

Pinterest



Reddit



Spotify



Instagram



Su nombre se inspira en el Guitarrista Django Reinhardt, del cual Adrian Holovaty, uno sus cocreadores, es admirador.



¿Por qué es relevante?



Productividad	Seguridad	Escalabilidad	Comunidad activa
Django ofrece una arquitectura sólida y componentes reutilizables que simplifican el desarrollo. Los desarrolladores pueden centrarse en la lógica de la aplicación en lugar de preocuparse por la infraestructura básica.	Django se preocupa por la seguridad y viene con características incorporadas para proteger las aplicaciones web contra las vulnerabilidades más comunes, como ataques de inyección de SQL y ataques CSRF.	Django es altamente escalable y se ha utilizado en aplicaciones de todos los tamaños, desde startups hasta grandes empresas. Esto lo convierte en una elección versátil para una variedad de proyectos.	.Django cuenta con una comunidad apasionada de desarrolladores que contribuyen a su crecimiento y mejora continua. Esto significa que siempre hay soporte y recursos disponibles.

¿Por qué usar Django?



Seguridad



ORM
(Mapeo
Objeto-Relaiconal)



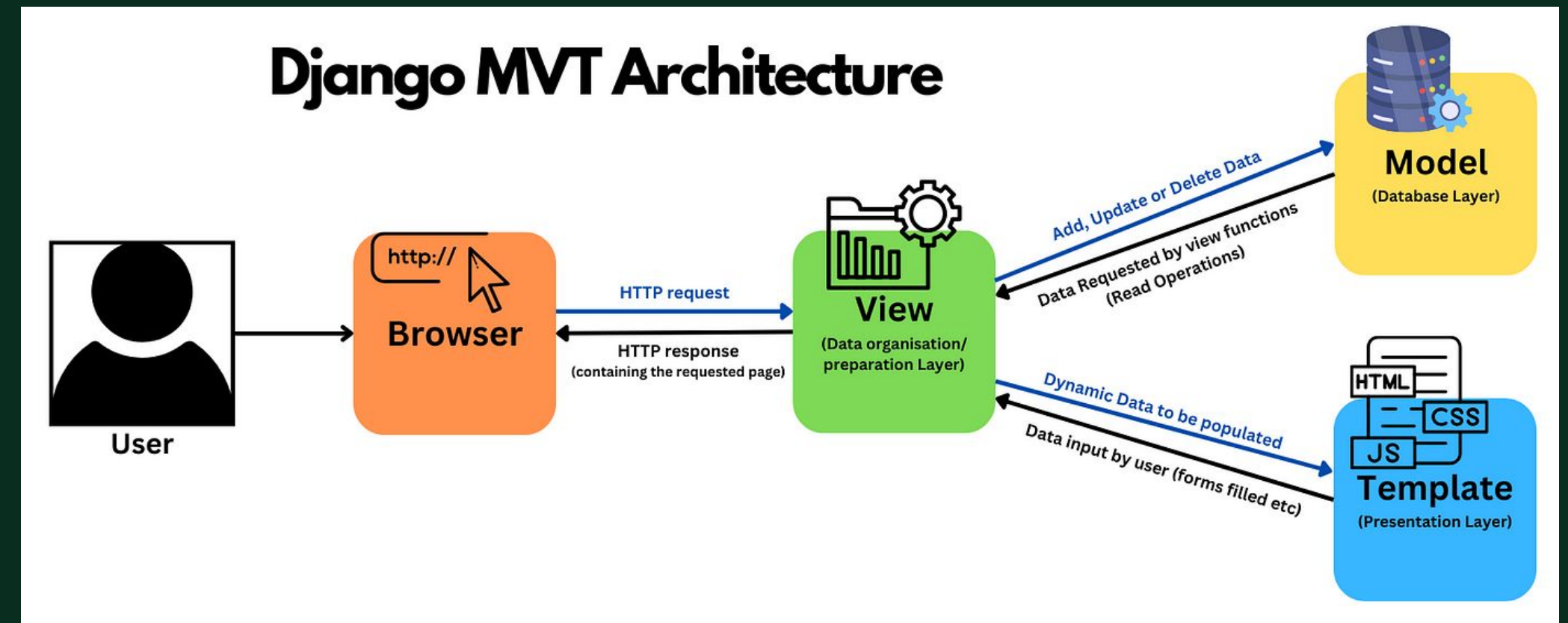
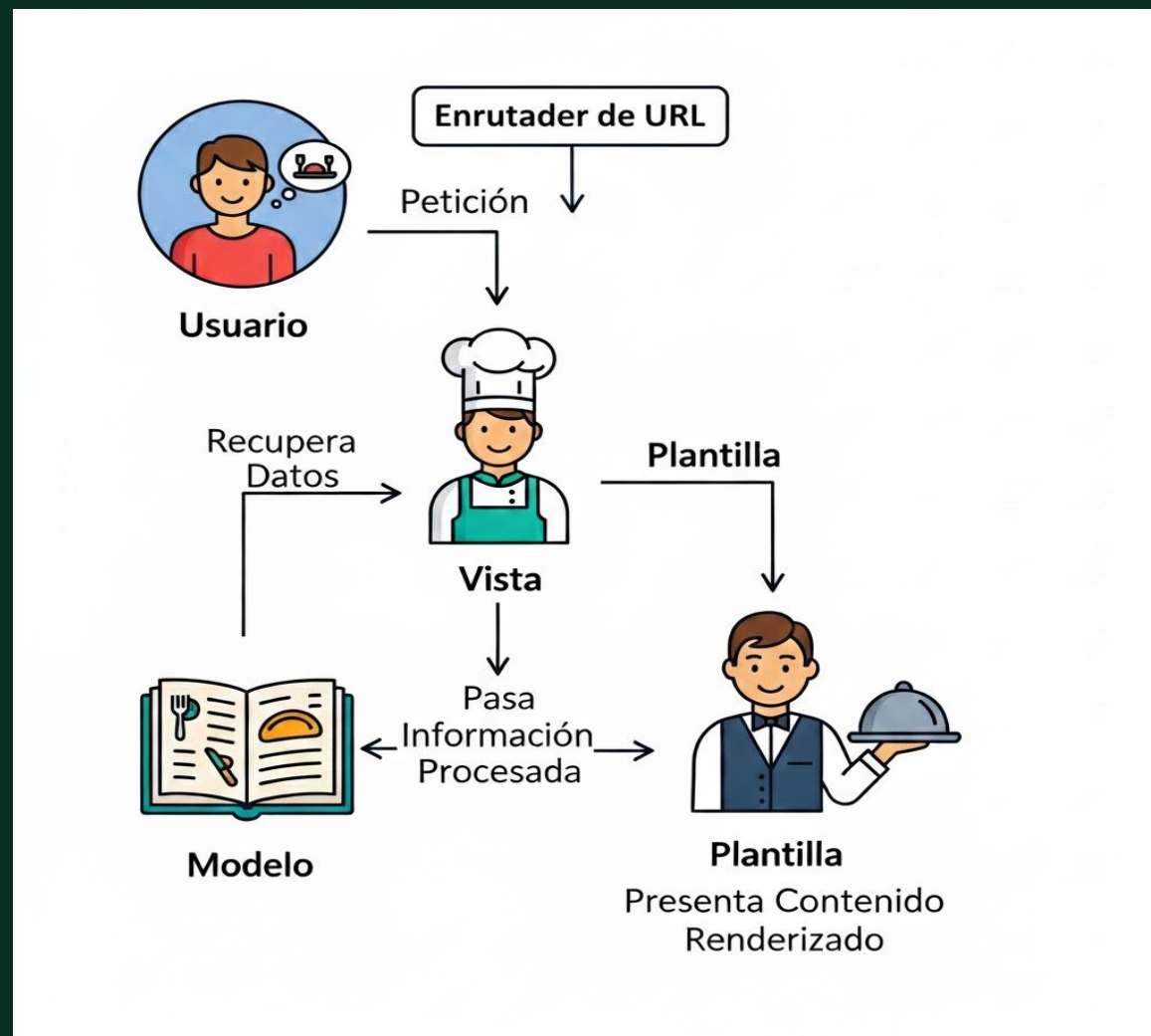
Admin
(Panel de
Administración)



Escalable



Arquitectura MTV



Imagina que vas a un restaurante y pides tu comida favorita. Tras bambalinas, el restaurante tiene un recetario que explica cómo preparar cada plato. El chef usa las recetas para preparar tu comida exactamente como la pediste. Una vez lista, el camarero te la sirve de forma presentable.

Al igual que un chef sigue la receta para crear el plato, la **Vista** utiliza el **Modelo** para recopilar y procesar datos. Finalmente, al igual que el camarero que sirve el plato, la **Plantilla** garantiza que el resultado final se presente de forma clara y atractiva para el usuario.



Requisitos del sistema



Python 3.8+

`pip3 --version`



Índice de Paquetes de Python

La herramienta pip viene incluida por defecto con las versiones de Python 3.4 y superiores



Entorno virtual de Python.

Navega hasta el directorio de tu proyecto y ejecuta el siguiente comando para crear un nuevo entorno virtual.

`python3 -m venv myenv`

`source myenv/Script/activate # Linux/macOS`

`env\Scripts\activate # Windows`



Instalación de Django

```
(myenv)
jdcad@JDCadenas MINGW64 ~/Desktop/DJANGO/curso-iujo
• $ pip install django
Collecting django
  Downloading django-5.2.6-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref>=3.8.1 (from django)
  Downloading asgiref-3.9.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.3.1 (from django)
  Downloading sqlparse-0.5.3-py3-none-any.whl.metadata (3.9 kB)
Collecting tzdata (from django)
  Using cached tzdata-2025.2-py2.py3-none-any.whl.metadata (1.4 kB)
Downloading django-5.2.6-py3-none-any.whl (8.3 MB)
 8.3/8.3 MB 7.3 MB/s 0:00:01
Downloading asgiref-3.9.1-py3-none-any.whl (23 kB)
Downloading sqlparse-0.5.3-py3-none-any.whl (44 kB)
Using cached tzdata-2025.2-py2.py3-none-any.whl (347 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.9.1 django-5.2.6 sqlparse-0.5.3 tzdata-2025.2
❖ (myenv)
jdcad@JDCadenas MINGW64 ~/Desktop/DJANGO/curso-iujo
○ $
```

ejecutar el siguiente comando en la consola. no te olvides de añadir el punto . al final

```
(myenv)
jdcad@JDCadenas MINGW64 ~/Desktop/DJANGO/curso-iujo
$ django-admin startproject mysite .
```

```
(myenv)
jdcad@JDCadenas MINGW64 ~/Desktop/DJANGO/curso-iujo
○ $ python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
September 09, 2025 - 00:06:06
Django version 5.2.6, using settings 'mysite.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.

For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
[09/Sep/2025 00:06:35] "GET / HTTP/1.1" 200 12068
Not Found: /favicon.ico
[09/Sep/2025 00:06:35] "GET /favicon.ico HTTP/1.1" 404 2208
□
```

Instalación de Django

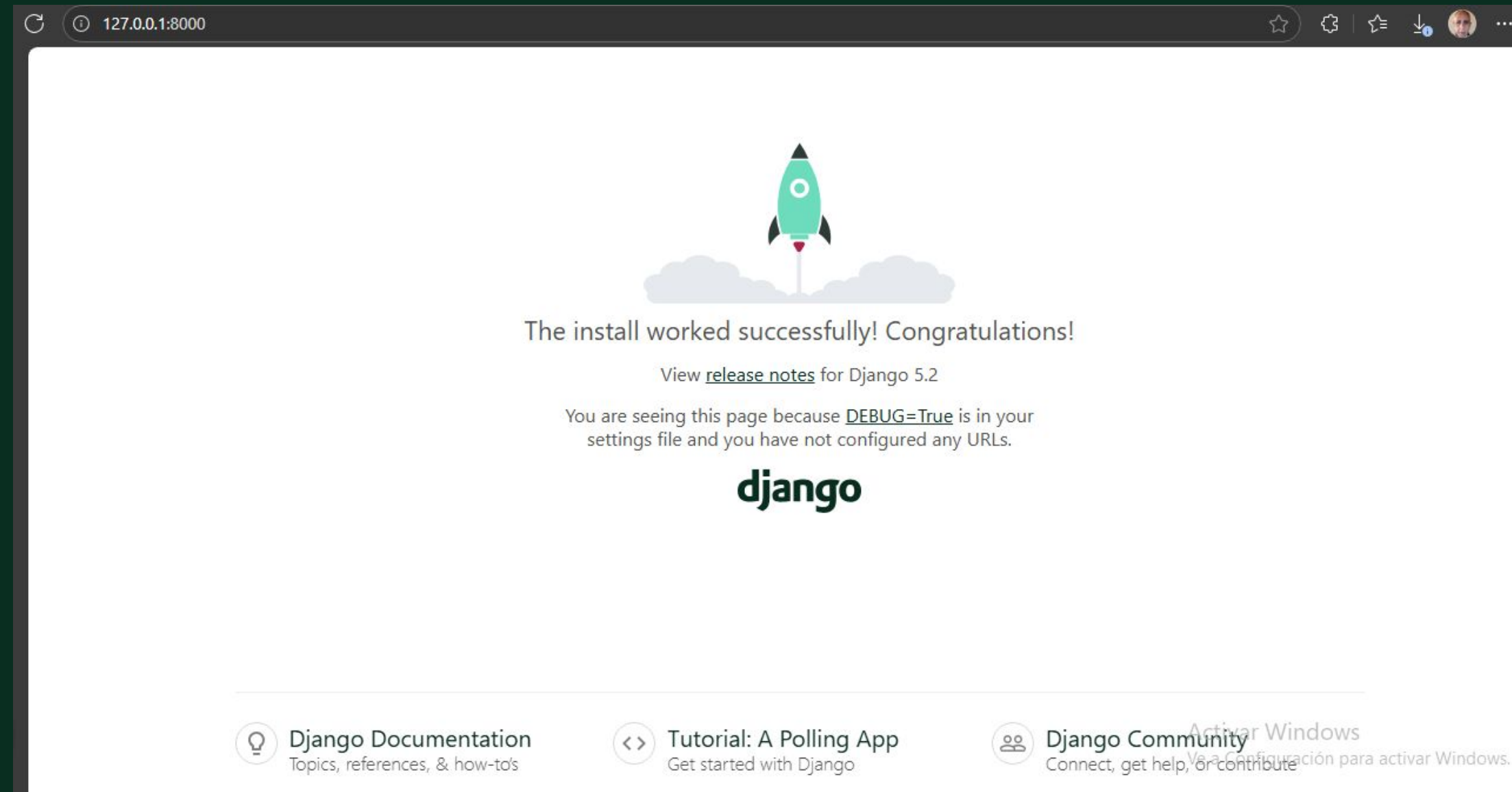
```
(myenv)
jdcad@JDCadenas MINGW64 ~/Desktop/DJANGO/curso-iujo
$ python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

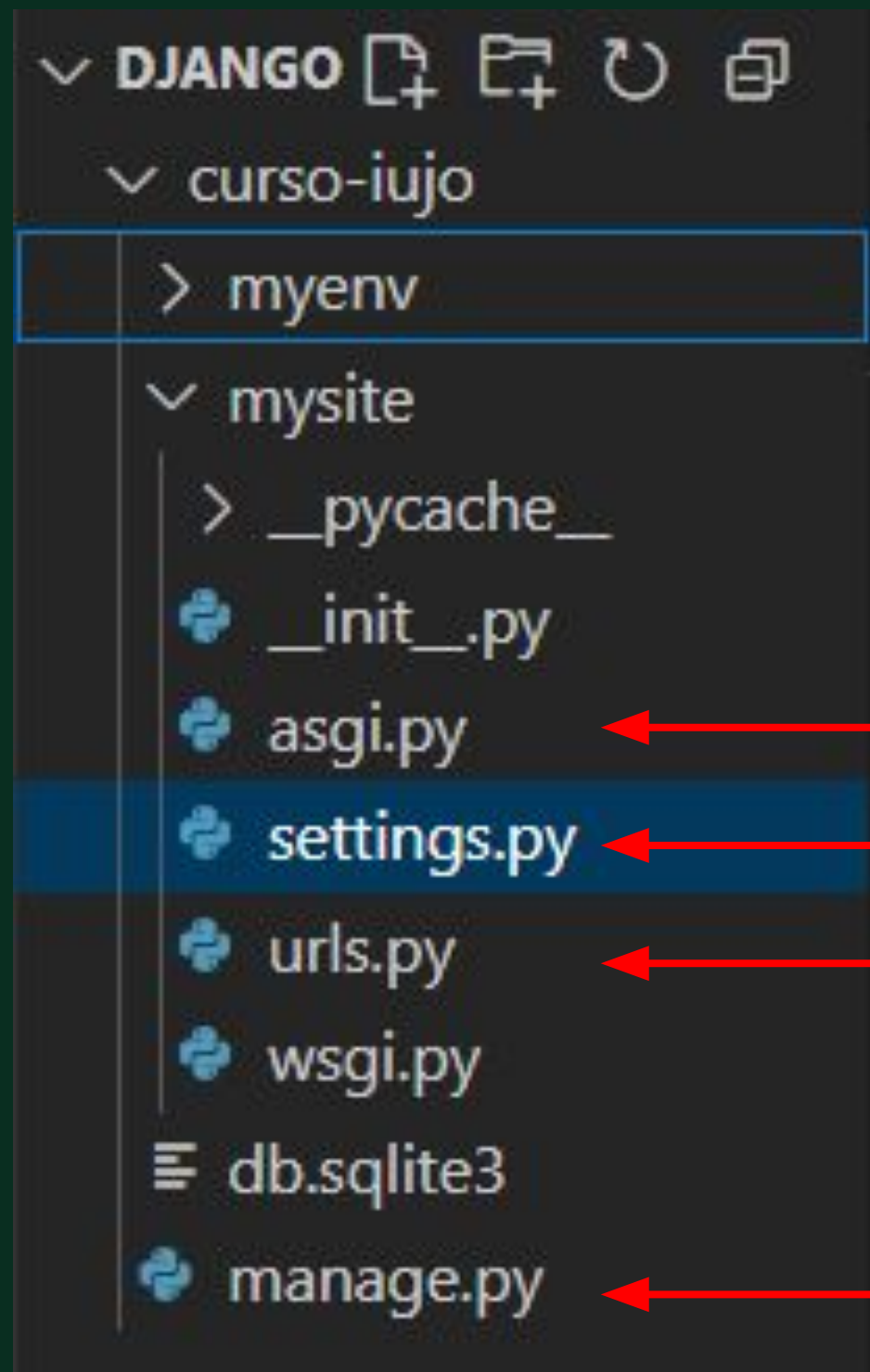
You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
September 09, 2025 - 00:06:06
Django version 5.2.6, using settings 'mysite.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
[09/Sep/2025 00:06:35] "GET / HTTP/1.1" 200 12068
Not Found: /favicon.ico
[09/Sep/2025 00:06:35] "GET /favicon.ico HTTP/1.1" 404 2208
□
```

Instalación de Django



Árbol de archivos de proyecto:



Este archivo es simplemente una interfaz de puerta de enlace usada para despliegues

Este archivo contiene la configuración de todo el proyecto.

Este archivo es el responsable de mapear las rutas y caminos (paths) en nuestro proyecto.

Un atajo para usar la utilidad de línea de comando django-admin.



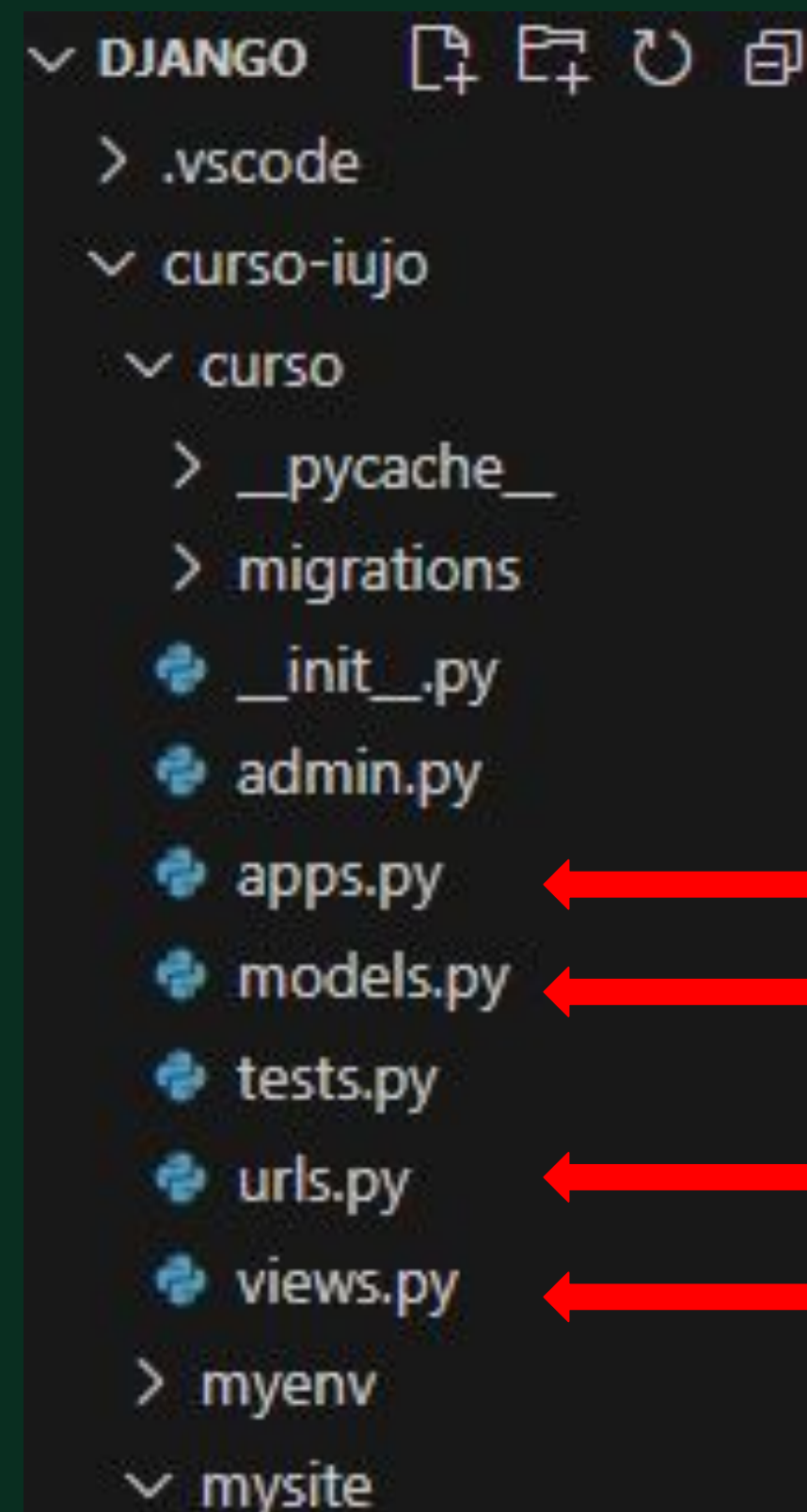


Reto final

Objetivo:

- Crear modulo aplicación

```
(myenv)
jdcad@JDCadenas MINGW64 ~/Desktop/DJANGO/curso-iujo
$ python manage.py startapp curso
```



Este es un archivo de configuración de la propia aplicación.

Aquí definimos las entidades de nuestra aplicación web.

Este archivo se encarga de mapear las rutas y caminos en nuestra app

Este es el archivo donde manejamos el ciclo de solicitud/respuesta de nuestra aplicación web.





urls.py

```
views.py  urls.py ...\mysite X  urls.py ...\curso
curso-iujo > mysite > urls.py > ...
1  """
2  URL configuration for mysite project.
3
4  The `urlpatterns` list routes URLs to views. For more information please see:
5  | https://docs.djangoproject.com/en/5.2/topics/http/urls/
6  Examples:
7  Function views
8  | 1. Add an import:  from my_app import views
9  | 2. Add a URL to urlpatterns:  path('', views.home, name='home')
10 Class-based views
11 | 1. Add an import:  from other_app.views import Home
12 | 2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
13 Including another URLconf
14 | 1. Import the include() function: from django.urls import include, path
15 | 2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
16 """
17 from django.contrib import admin
18 from django.urls import include, path
19
20 urlpatterns = [
21     path('admin/', admin.site.urls),
22     path('', include('curso.urls')),
23 ]
```



02 Modelos y bases de datos

ORM, migraciones, consultas, shell



Objetivos:

-  Definir modelos
-  Aplicar migraciones
-  Consultas con ORM

¿Qué es el ORM?



ORM significa Object-Relational Mapping (Mapeo Objeto-Relacional).

Es una técnica de programación que permite convertir datos entre sistemas incompatibles, como un lenguaje orientado a objetos (como Python) y una base de datos relacional (como SQLite o PostgreSQL).

En otras palabras, el ORM te permite trabajar con la base de datos usando código Python, sin necesidad de escribir consultas SQL directamente.



Ejemplo práctico con el ORM

Modelo Producto:

```
class Producto(models.Model):
    nombre = models.CharField(max_length=100)
    precio = models.DecimalField(max_digits=6, decimal_places=2)
    descripcion = models.TextField()
```

Con el ORM, puedes hacer cosas como:

ACCIÓN	CÓDIGO EN DJANGO(ORM)
Obtener todos los productos	Producto.objects.all()
Filtrar por precio mayor a 50	Producto.objects.filter(precio__gt=50)
Obtener un producto por ID	Producto.objects.get(id=1)
Crear un nuevo producto	Producto.objects.create(nombre="Laptop", precio=800)
Actualizar un producto	prod = Producto.objects.get(id=1) prod.precio = 900 prod.save()
Eliminar un producto	prod.delete()

- Crear el modelo Producto
- Hacer migraciones (makemigrations, migrate)
- Abrir el shell: python manage.py shell
- Ejecutar consultas ORM y ver resultados
- Comparar con SQL (opcional): usar str(queryset.query) para ver el SQL generado



Ejemplo práctico con el ORM

Modelo Categoria:

```
categoria = models.ForeignKey(Categoria,on_delete=models.CASCADE)
```

```
class Producto(models.Model):  
    nombre = models.CharField(max_length=100)  
    precio = models.DecimalField(max_digits=6, decimal_places=2)  
    descripcion = models.TextField()  
    categoria = models.ForeignKey(Categoria,  
                                on_delete=models.CASCADE,default=1)
```

Crear el modelo Categoria

Hacer migraciones (makemigrations, migrate)

Abrir el shell: `python manage.py shell`

Ejecutar consultas ORM y ver resultados

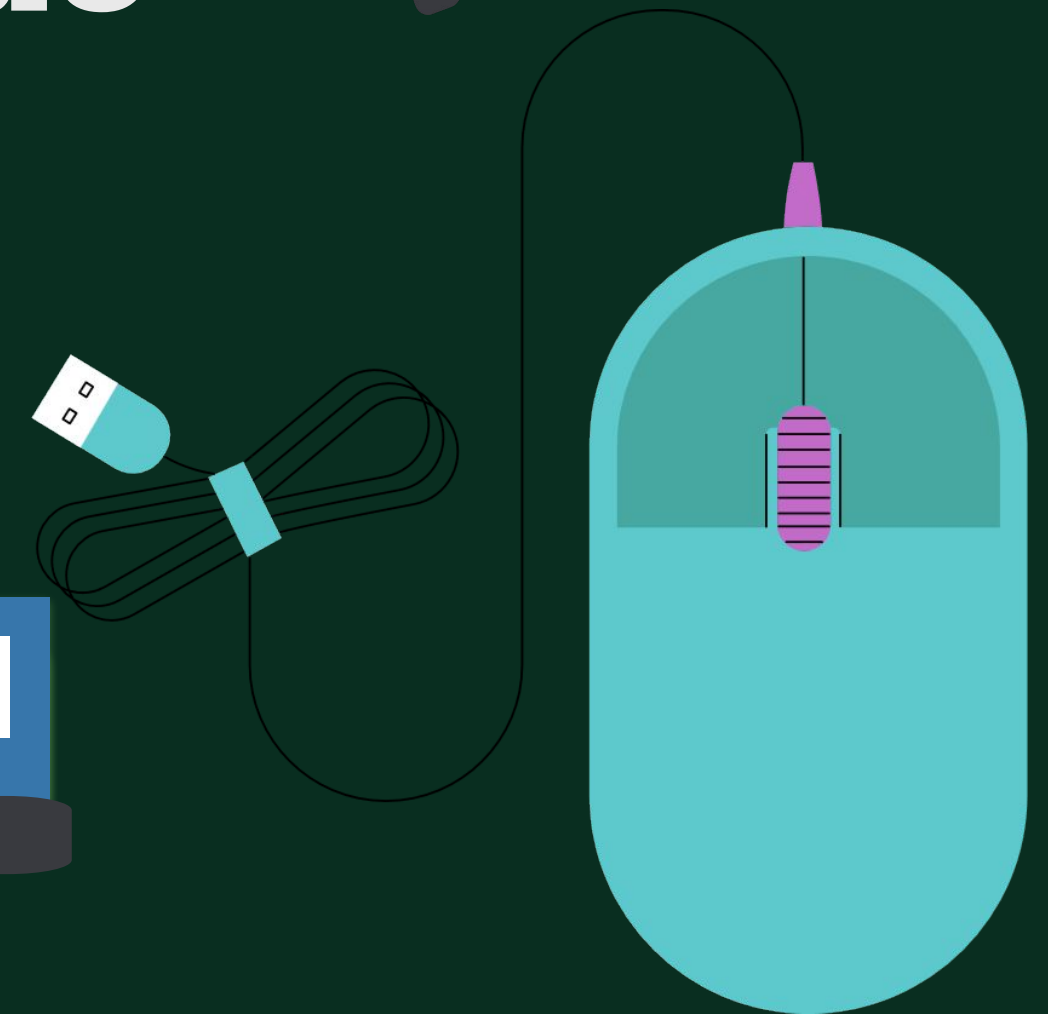
Comparar con SQL (opcional): usar
`str(queryset.query)` para ver el SQL generado

03

Vistas, URLs y plantillas



Tu Primer desarrollo web Profesional



Objetivos:

-  **Crear vistas FBV y CBV**
-  **Usar plantillas con herencia**
-  **Implementar formularios y rutas**

FBV vs CBV:



VISTAS BASADAS EN FUNCIONES (FBV)	VISTAS BASADAS EN CLASES (CBV)
✓ Ventajas • Fáciles de entender, ideales para principiantes. • Perfectas para lógica simple o personalizada. • Más intuitivas si vienes de programación básica. ✗ Desventajas: • Se repite mucho código (por ejemplo, verificar métodos POST, renderizar plantillas, etc.). • No aprovechan la herencia ni la reutilización como las clases. • Pueden volverse largas y difíciles de mantener en proyectos grandes.	✓ Ventajas • Reutilizables: puedes heredar y extender comportamientos. • Menos código repetido: Django ya incluye lógica común. • Vistas genéricas poderosas: ○ ListView: lista objetos automáticamente. ○ DetailView: muestra un objeto por su ID. ○ CreateView: crea formularios y guarda datos. ○ UpdateView, DeleteView, etc. ✗ Desventajas: • Curva de aprendizaje más alta (requiere conocimientos de POO). • Pueden parecer "mágicas" al principio (el comportamiento viene de la herencia). • Menos flexibles para lógica muy personalizada (aunque se pueden sobrescribir métodos).

FBV vs CBV:



Característica	FBV	CBV
Sintaxis	<code>def mi_vista(request)</code>	<code>class MiVista(View):</code>
Uso en URLs	<code>path('ruta/', mi_vista)</code>	<code>path('ruta/', MiVista.as_view())</code>
Ideal para	Lógica simple o personalizad	CRUD, listados, formularios comunes
Ejemplos comunes	Login personalizado, APIs simples	Listar productos, crear usuarios



Ejemplo práctico apps

Estructura modular en Django, se sigue el principio:
"Una app, una funcionalidad clara"

Reto práctico

"Crear app de registro de estudiantes con formulario y vista"

Pasos: modelo → vista → template → formulario





Ejemplo práctico apps

Por ejemplo:

- `estudiantes` → maneja todo lo relacionado con estudiantes
- `profesores` → maneja profesores
- `cursos` → maneja asignaturas
- `matriculas` → maneja inscripciones

Cada app tiene su propia estructura:

```
1 estudiantes/  
2 |— models.py    → Definición de modelos (Estudiante, etc.)  
3 |— views.py     → Lógica de las vistas  
4 |— urls.py      → Rutas específicas de esta app  
5 |— forms.py     → Formularios  
6 |— admin.py     → Configuración del panel de administración  
7 |— templates/   → Plantillas HTML para esta app  
8 |— migrations/  → Archivos de migración de base de datos
```

Se crea con:

```
bash  
1 python manage.py startapp estudiantes
```

1. Crear la app:

```
bash  
1 python manage.py startapp estudiantes
```

2. Definir modelo (`estudiantes/models.py`):

```
python  
1 from django.db import models  
2  
3 class Estudiante(models.Model):  
4     nombre = models.CharField(max_length=100)  
5     email = models.EmailField()  
6     edad = models.IntegerField()  
7  
8     def __str__(self):  
9         return self.nombre
```



Ejemplo práctico apps

Estructura modular

3. Crear vistas (`estudiantes/views.py`):

```
python
1 from django.shortcuts import render
2 from .models import Estudiante
3
4 def lista_estudiantes(request):
5     estudiantes = Estudiante.objects.all()
6     return render(request, 'estudiantes/lista.html', {'estudiantes': estudiantes})
```

4. Agregar rutas (`estudiantes/urls.py`):

```
python
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path('', views.lista_estudiantes, name='lista_estudiantes'),
6 ]
```

5. Conectar al proyecto (`mi_proyecto/urls.py`):

```
python
1 from django.urls import path, include
2
3 urlpatterns = [
4     path('estudiantes/', include('estudiantes.urls')),
5 ]
```



Ejemplo práctico apps

Plantilla con herencia

Crea `templates/base.html` :

```
html
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>{% block title %}Curso Django{% endblock %}</title>
5 </head>
6 <body>
7   <header>
8     <h1>🎓 Registro de Estudiantes</h1>
9   </header>
10  <main>
11    {% block content %}
12    {% endblock %}
13  </main>
14 </body>
15 </html>
```

Crea `templates/formulario.html` :

```
html
1 {% extends 'base.html' %}
2
3 {% block content %}
4   <h2>Registra un estudiante</h2>
5   <form method="post">
6     {% csrf_token %}
7     {{ form.as_p }}
8     <button type="submit">Guardar</button>
9   </form>
10  {% endblock %}
```

🔗 Paso 6: Configurar URLs

En `mi_proyecto/urls.py` :

```
python
1 from django.urls import path
2 from estudiantes import views
3
4 urlpatterns = [
5     path('formulario/', views.formulario_estudiante, name='formulario'),
6     # ... otras rutas
7 ]
```

Asegúrate de configurar `TEMPLATES` en `settings.py` :

```
python
1 'DIRS': [BASE_DIR / 'templates'],
```



Ejemplo práctico apps Formulario

Genera y aplica migraciones:

bash

```
1 python manage.py makemigrations
2 python manage.py migrate
```

Paso 3: Formulario con ModelForm

Crea `estudiantes/forms.py` :

python

```
1 from django import forms
2 from .models import Estudiante
3
4 class EstudianteForm(forms.ModelForm):
5     class Meta:
6         model = Estudiante
7         fields = ['nombre', 'email']
```

Paso 4: Vista para el formulario

En `estudiantes/views.py` :

python

```
1 from django.shortcuts import render, redirect
2 from .forms import EstudianteForm
3
4 def formulario_estudiante(request):
5     if request.method == 'POST':
6         form = EstudianteForm(request.POST)
7         if form.is_valid():
8             form.save()
9             return redirect('formulario') # Redirige al mismo formulario
10    else:
11        form = EstudianteForm()
12
13    return render(request, 'formulario.html', {'form': form})
```

04

Autenticación y seguridad

Protege tu aplicación desde el primer acceso



django



Objetivos:

-  **Login/logout/registro**
-  **Decoradores**
-  **Permisos y seguridad**

Sistema de usuarios

login

logout

Registro

DJANGO

> .vscode

> curso-iujo

> accounts

> __pycache__

> migrations

> templates\registration

> login.html

> register.html

> __init__.py

> admin.py

> apps.py

> models.py

> tests.py

> views.py

> curso

> estudiantes

> myenv

> mysite

> __pycache__

> __init__.py

> asgi.py

> settings.py

> urls.py

> wsgi.py

> db.sqlite3

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Document</title>
</head>
<body>
  <h1>Login</h1>
  <form method="POST" action="{% url 'login' %}" ">
    {% csrf_token %}
    {{ form.as_div }}
    <button type="submit">Iniciar sesión</button>
  </form>
</body>
</html>
```

estudiantes

> __pycache__

> migrations

> templates\estudiantes

> base.html

> formulario.html

> listado.html

```
<html lang="en">
</head>
<body>
  <header> Estudiantes </header>
  <nav>
    {% if user.is_authenticated %}
      Hola, {{ user.username }}!
      <form method="post" action="{% url 'logout' %}" style="display: inline-block;">
        {% csrf_token %}
        <button type="submit" style="border: none; background: none; color: red;">
          Cerrar Sesión
        </button>
      </form>
    {% else %}
      <a href="{% url 'login' %}">Iniciar Sesión</a>
    {% endif %}
  </nav>
  <main>
    {% block content %}
    {% endblock content %}
  </main>
</body>
</html>
```

curso-iujo > accounts > urls.py > ...

```
1 from django.urls import path
2
3 from .views import SignUpView
4
5
6 urlpatterns = [
7     path("signup/", SignUpView, name="signup"),
8 ]
```

curso-iujo > accounts > views.py > ...

```
30 # VISTA DE REGISTRO BASADA EN FUNCION (VBF)
31 # -----
32 def SignUpView(request):
33     # es POST (el usuario envió el formulario)
34     if request.method == 'POST':
35         form = UserCreationForm(request.POST)
36         # Validar el formulario
37         if form.is_valid():
38             # Guardar el nuevo usuario en la base de datos
39             user = form.save()
40             username = form.cleaned_data.get('username')
41             password = form.cleaned_data.get('password1')
42             # Autenticar y iniciar sesión al usuario automáticamente (opcional)
43             # from django.contrib.auth import authenticate, login
44             user = authenticate(username=username, password=password)
45             # Opcional: Iniciar sesión automáticamente al usuario
46             # Si quieres que se redirija al login: return redirect('login')
47             login(request, user)
48             # Redirigir al usuario a la página de inicio (o a LOGIN_REDIRECT_URL)
49             return redirect('estudiantes:listado_estudiantes')
50
51 # Si la petición es GET (mostrar el formulario)
52 else:
53     form = UserCreationForm()
54     # Renderizar el template con el formulario
55     return render(request, 'registration/register.html', {'form': form})
```

Sistema de usuarios

login

1. Configuración Inicial: URLs

Añadir `path('accounts/', include('django.contrib.auth.urls'))` en `urls.py`.

2. Configuración Adicional: settings.py

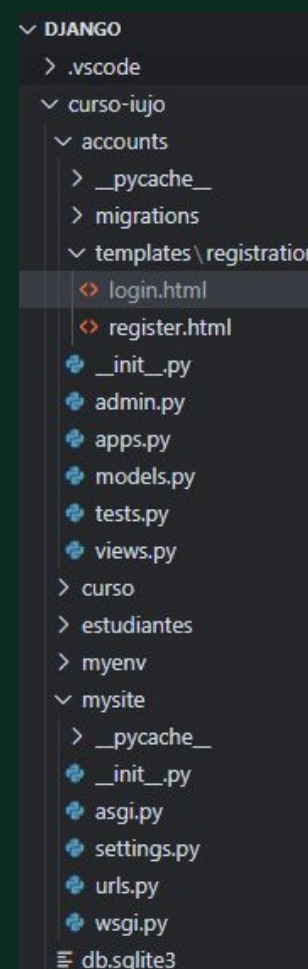
Poner `LOGIN_REDIRECT_URL = '/'` en `settings.py`.

3. Creación de Templates

Crear el archivo `login.html` dentro de `templates/registration/` con un formulario que use `{% csrf_token %}` y `{{ form.as_div }}`.

4. Mostrar el Estado de Autenticación

Usar `{% if user.is_authenticated %}` en los templates para mostrar contenido personalizado.



```
curso-iujo > accounts > templates > registration > login.html > html
2  <html lang="en">
3  <head>
4
5  </head>
6
7  </head>
8  <body>
9      <h1>Login</h1>
10     <form method="POST" action="{% url 'login' %}" >
11
12         {% csrf_token %}
13         {{ form.as_div }}
14         <button type="submit">Iniciar sesión</button>
15     </form>
16     <p>
17         ¿No tienes una cuenta? <a href="{% url 'signup' %}">Regístrate aquí</a>
18     </p>
19
20 </body>
21 </html>
```



Sistema de usuarios

logout

1. El Enlace de Logout en Template

Dentro de cualquiera de tus templates (normalmente en la barra de navegación o en la base (base.html)), crea un enlace que apunte a la vista de cierre de sesión.

La vista de logout de Django se registra automáticamente con el nombre de URL 'logout' (porque está incluida dentro de django.contrib.auth.urls).

2. Configuración de Redirección (Opcional, pero Recomendado)

Por defecto, después de cerrar sesión, Django te redirigirá a la página /accounts/logged_out/. Si quieres redirigir al usuario a la página de login o a la página principal, puedes configurarlo en settings.py:

```
# La URL a la que se redirige después que el usuario CIERRA sesión.  
# Puedes usar:  
# '/accounts/login/' (vuelve al formulario de login)  
# '/' (vuelve a la página principal)  
LOGOUT_REDIRECT_URL = '/accounts/login/'
```

Con esto, cada vez que un usuario haga clic en el enlace "Cerrar Sesión"

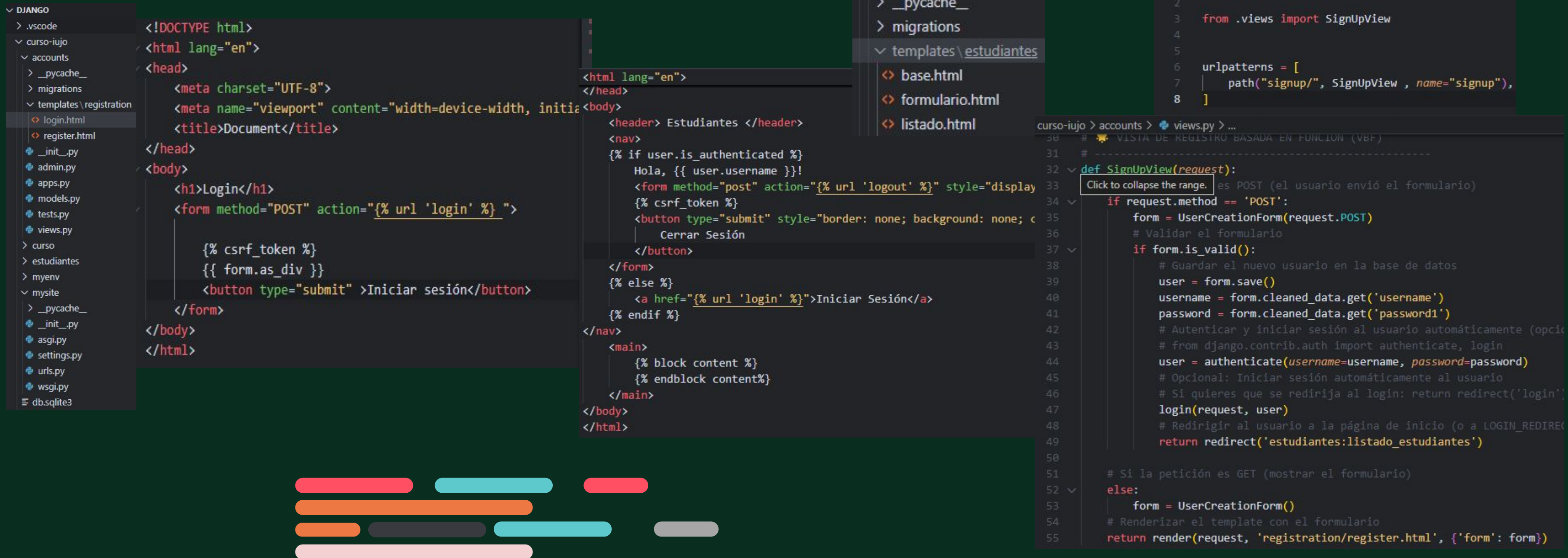
```
<html lang="en">  
</head>  
<body>  
  <header> Estudiantes </header>  
  <nav>  
    {% if user.is_authenticated %}  
      Hola, {{ user.username }}!  
      <form method="post" action="{% url 'logout' %}" style="display:  
        {% csrf_token %}  
        <button type="submit" style="border: none; background: none; c  
          | Cerrar Sesión  
        </button>  
      </form>  
    {% else %}  
      <a href="{% url 'login' %}">Iniciar Sesión</a>  
    {% endif %}  
  </nav>  
  <main>  
    {% block content %}  
    {% endblock content%}  
  </main>  
</body>  
</html>
```

Sistema de usuarios

login

logout

Registro



05

API REST con Django REST Framework



POSTMAN

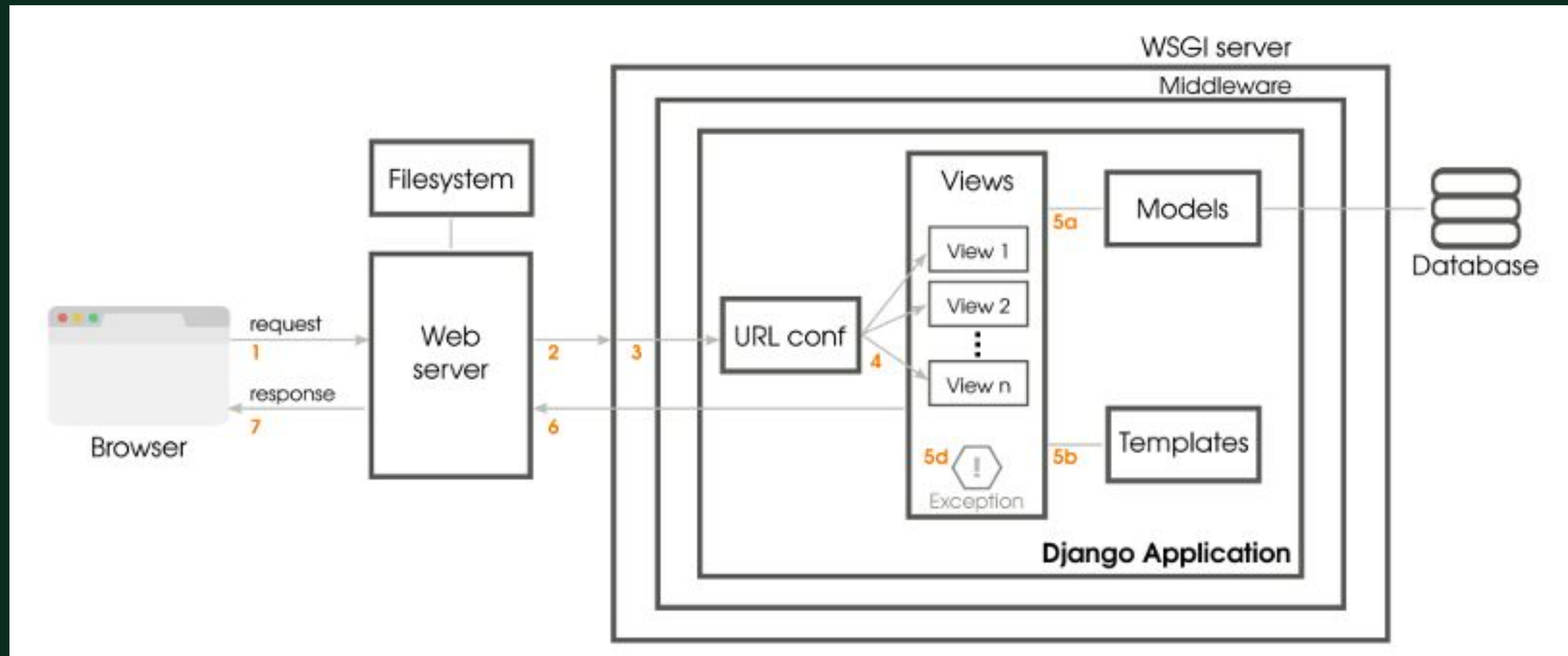


Objetivos:



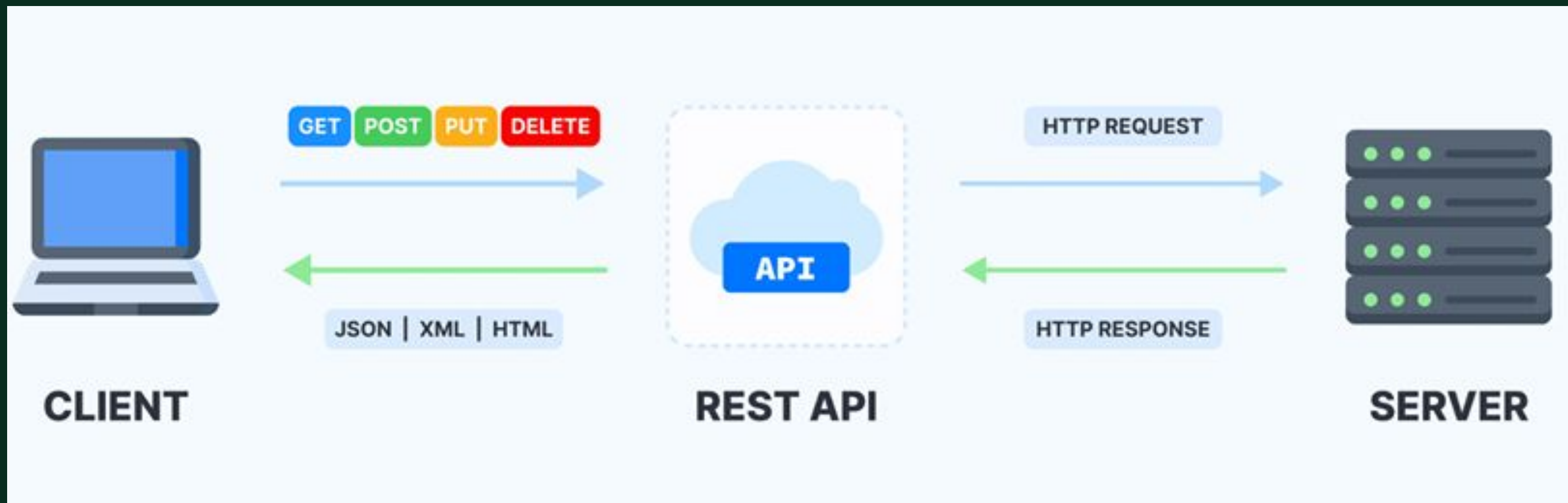
Exponer datos mediante una API

Django REST Framework



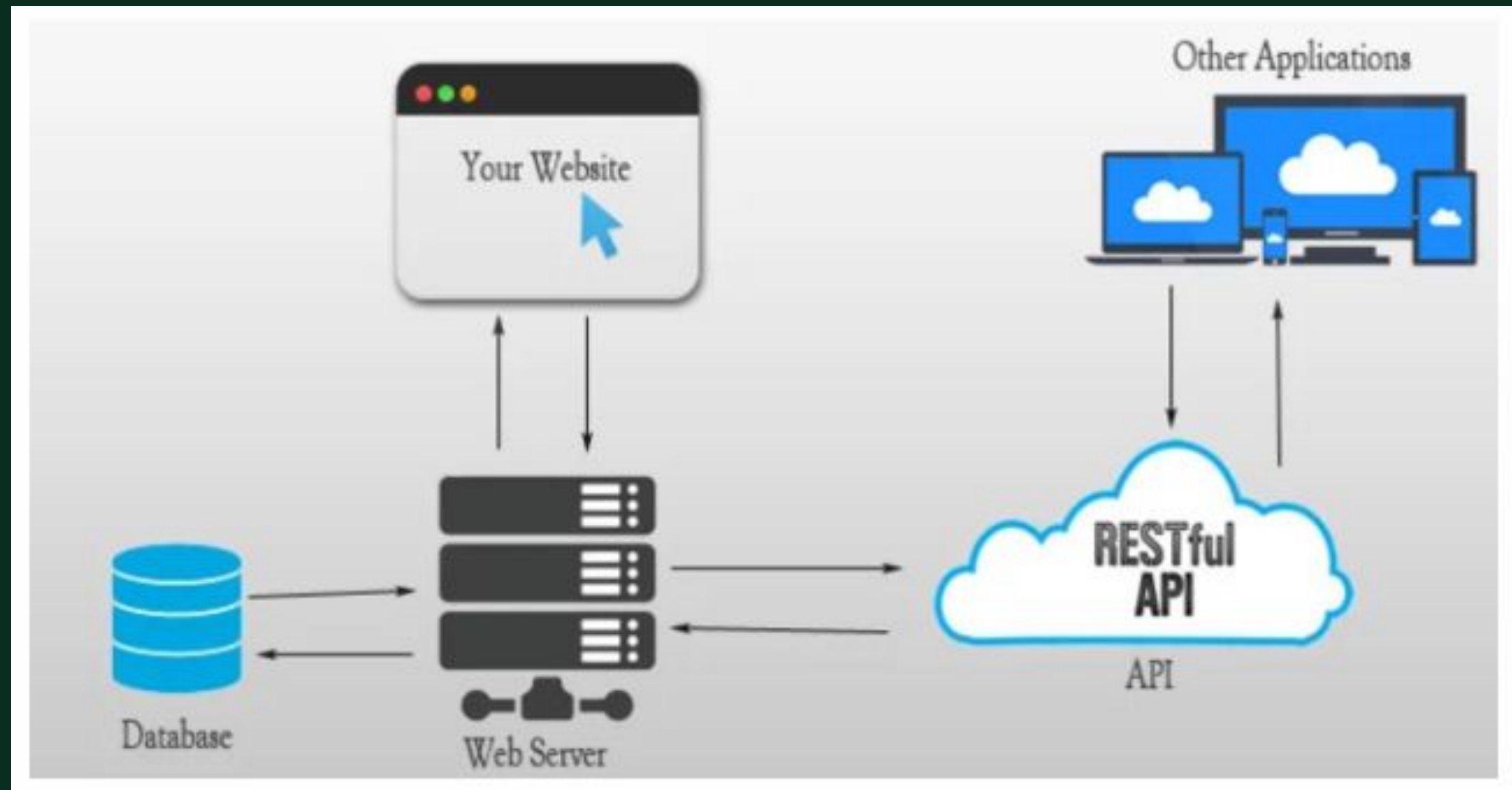
Django REST Framework

Rest Api Model



Django REST Framework

Modelos Combinados



Django REST Framework

Instalación

en ambiente virtual de django (recomendado en requirements.txt)

```
pip install djangorestframework
```

settings.py

```
INSTALLED_APPS = [  
    ...  
    'rest_framework',  
]
```

urls.py

```
urlpatterns = [  
    ...  
    path('api-auth/', include('rest_framework.urls'))  
]
```

Incluye las url django authentication para el framework rest.
Por defecto configurado en BasicAuthentication y
SessionAuthentication



Django REST Framework

Instalación

bash

```
1 pip install djangorestframework
```

- Añadir a `INSTALLED_APPS` :

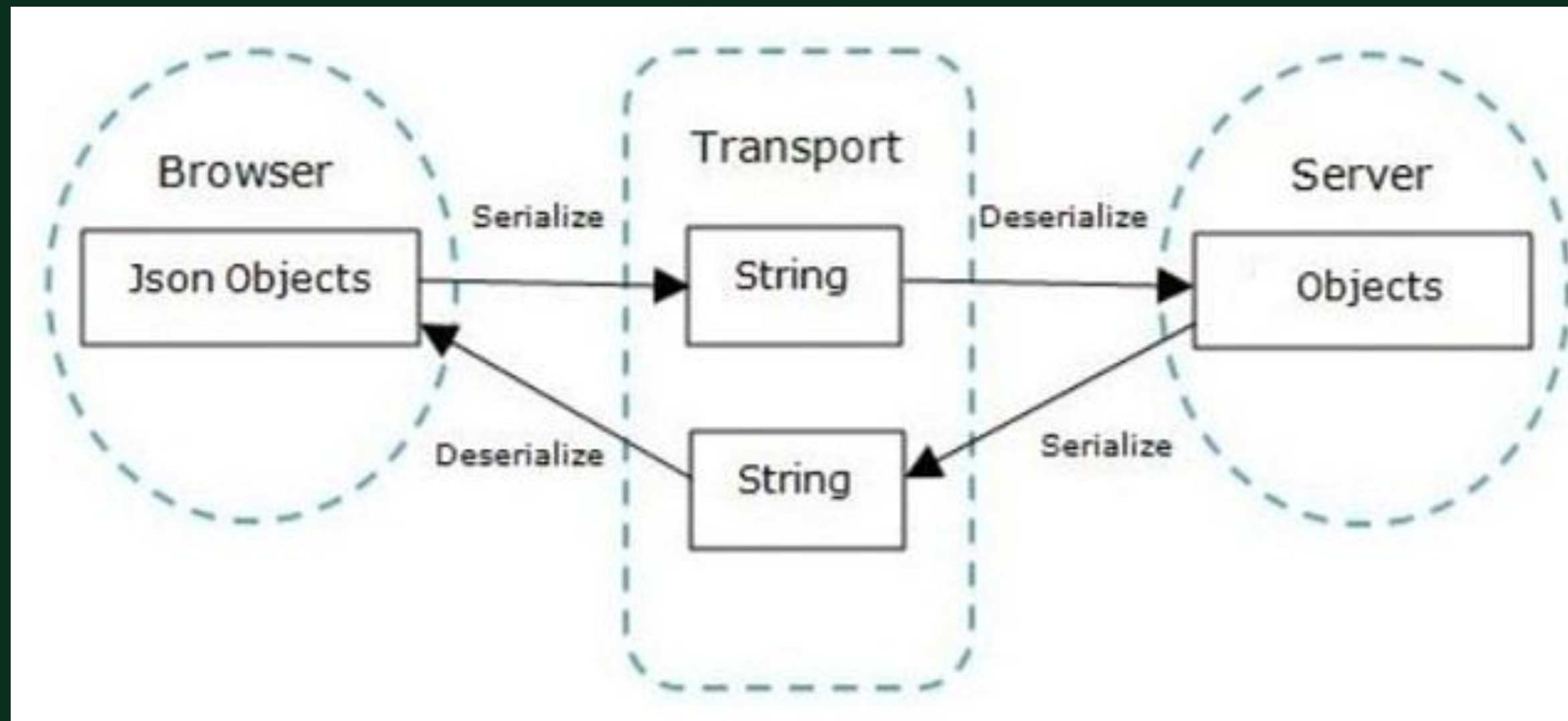
python

```
1 'django.contrib.admin',  
2 'rest_framework',
```



Django REST Framework

Serialización



Django REST Framework

Serialización

- Convertir objetos de Python (modelos) en JSON y viceversa.
- Permite que Django entienda cómo enviar/recibir datos en formato JSON.



Django REST Framework

Ejemplo Práctico

```
django-admin startapp cac_api
```

```
▼ cac_api
  > migrations
  + __init__.py
  + admin.py
  + apps.py
  + models.py
  + serializers.py
  + tests.py
  + urls.py
  + views.py
```

Si utilizamos una arquitectura de microservicios, debería poseer sus propios modelos, vistas y urls.

Si la api, es parte de un website, acoplamos dicha api a la app que posee los modelos con los que interactuamos



Django REST Framework

Serialización con Serializer y ModelSerializer

Serializers.py

```
from rest_framework import serializers  
from cac.models import EstudianteM
```

Como django provee Form y
ModelForm, REST provee Serializer y
ModelSerializer

```
class EstudianteMSerializer(serializers.ModelSerializer):  
    class Meta:  
        model = EstudianteM  
        fields = ['id', 'nombre_m', 'apellido_m', 'email_m', 'dni_m', 'matricula_m']
```



Django REST Framework

Serialización con Serializer y ModelSerializer

Views.py (ViewSet)

—————> Son Vistas Basadas en Clases

```
from rest_framework import viewsets
from rest_framework import permissions
from cac.models import EstudianteM
from cac_api import serializers

# Create your views here.
class EstudianteMViewSet(viewsets.ModelViewSet):
    queryset = EstudianteM.objects.all()
    serializer_class = serializers.EstudianteMSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly]
```



Django REST Framework

Serialización con Serializer y ModelSerializer

```
path('api-auth/', include('rest_framework.urls', namespace='rest_framework')),
```

```
from django.urls import include, path
```

```
from . import views
```

```
from rest_framework.routers import DefaultRouter
```

```
router = DefaultRouter()
```

```
router.register(r'estudiantes', views.EstudianteViewSet, basename='estudiante')
```

```
app_name = 'estudiantes'
```

```
urlpatterns = [
```

```
    path('api/', include(router.urls)),
```

```
    path('', views.listado_estudiantes, name='listado_estudiantes' ),
```

```
    path('formulario', views.formulario_estudiante, name='formulario_estudiante'),
```

```
]
```

